ЖёПка

Язык функционального программирования

Русский синтаксис · Вложенность через «3» · Самоимпорт файла

ОФИЦИАЛЬНАЯ СПЕЦИФИКАЦИЯ

Версия	1.0.0
Парадигма	Функциональная
Расширение	.ж
Ключевых слов	47
Типов данных	12 (8 встроенных + 4 специальных)
Примеров кода	22
Статус	Stable
Автор	Z.ai

ЧАСТЬ ДАТАСЕТА ИЗ 8 ФАЙЛОВ

```
01_introduction.md   — Введение в язык
02_faq.txt           — Часто задаваемые вопросы
03_operators.csv     — Таблица операторов Python → ЖёПка
04_grammar.json      — Формальная BNF-грамматика
05_tutorial.html     — Туториал с примерами
06_examples.docx     — Сборник примеров кода
07_tables.xlsx       — Детальные таблицы
08_specification.pdf — Этот документ
```

© Z.ai · Лицензия CC BY 4.0 · Спецификация свободна для использования

1. Введение

ЖёПка — это функциональный язык программирования, в котором исходный код полностью пишется на русском языке. Язык спроектирован с целью доказать, что программирование не обязано использовать английский синтаксис, а понятность кода достигается не отступами и скобками, а строгими правилами лексической вложенности. Главным отличием ЖёПки от других русифицированных языков является отказ от классических механизмов структурирования: в нём нет ни фигурных скобок, ни значимых отступов в духе Python. Вместо этого каждый новый уровень вложенности обозначается символом «З», повторённым соответствующее количество раз в начале строки.

Язык носит шуточное название, однако его дизайн подчинён строгим принципам функционального программирования: иммутабельные данные по умолчанию, функции как объекты первого класса, обширная поддержка высшего порядка, ленивые вычисления для последовательностей и явное управление контекстом через связывание «пусть». ЖёПка не претендует на продакшен-применимость; его основное назначение — исследование альтернативных синтаксических соглашений и обучение основам функционального мышления в привычной русскоязычной среде.

Каждая программа на ЖёПке начинается с обязательной строки самоимпорта «жёпка принадлежит <имя_файла>.ж». Эта строка не имеет аналогов в других языках: она буквально импортирует текущий файл в самого себя. Без неё интерпретатор откажется запускать программу, поскольку в философии языка файл должен осознавать собственную идентичность до того, как начнёт определять какую-либо логику. Это правило введено намеренно: оно подчёркивает, что в функциональной парадигме модуль — это не просто контейнер для кода, а самостоятельная сущность со своим именем и контекстом.

> **СОГЛАШЕНИЕ О ЯЗЫКАХ**
> Этот PDF следует правилу разделения контента: русские описания языка, операторов и грамматики на русском языке; примеры кода с английскими комментариями. Код пишется на русском, поскольку весь синтаксис ЖёПки — русский, но комментарии к нему даны на английском для международной аудитории.

2. Философия и принципы дизайна

ЖёПка построена на четырёх фундаментальных принципах, которые определяют все синтаксические и семантические решения языка. Эти принципы не являются взаимоисключающими; наоборот, они усиливают друг друга и формируют целостную концепцию, отличающую ЖёПку от других функциональных языков.

2.1. Принцип первый — русификация синтаксиса

Все ключевые слова языка являются дословным переводом соответствующих операторов Python. Это означает, что print становится «печать», if — «если», def — «функция», return — «вернуть», lambda — «лямбда», map — «карта», и так далее. Такая прямая трансляция снижает порог входа для русскоязычных разработчиков, уже знакомых с Python, и одновременно делает код самодокументируемым для тех, кто читает его без предварительной подготовки. При этом имена переменных и функций могут использовать как кириллицу, так и латиницу — язык не навязывает соглашения об именовании, ограничивая лишь множество зарезервированных слов.

2.2. Принцип второй — отказ от скобочной структуры

В ЖёПке отсутствуют фигурные скобки, квадратные скобки для индексации также заменены вызовами функций, а круглые скобки для группировки выражений используются минимально. Вместо этого применяется лексическая вложенность через символ «З»: количество троек в начале строки однозначно определяет уровень вложенности данного выражения в синтаксическом дереве. Это решение вдохновлено значимыми отступами Python, но вместо пробелов используется явный числовой маркер, что делает структуру визуально заметной даже в моноширинном редакторе без подсветки.

2.3. Принцип третий — обязательный самоимпорт

Первая строка каждого файла должна иметь вид «жёпка принадлежит <имя>.ж». Эта строка не является декоративной: она реализует механизм рефлексии на уровне файловой системы. Файл импортирует сам себя как модуль, что позволяет ему обращаться к собственным публичным определениям через имя модуля, а также гарантирует, что интерпретатор корректно разрешит циклические зависимости (поскольку файл знает о своём существовании с самого начала загрузки). Без этой строки запуск невозможен — интерпретатор вернёт ошибку «ОшибкаСамоИмпорта».

2.4. Принцип четвёртый — функциональная чистота по умолчанию

Все привязки «пусть» неизменяемы; для изменения состояния требуется явное использование «ячейка» (аналог ref в ML-языках). Побочные эффекты изолированы в монадоподобных конструкциях «эффект», а ввод-вывод осуществляется через «печать» и «открыть», которые помечены как эффектные операции. Такая дисциплина заимствована у Haskell и OCaml и адаптирована под русскоязычную аудиторию без введения сложной системы типов.

3. Самоимпорт файла

Строка «жёпка принадлежит <имя>.ж» — это не просто формальность. Она реализует механизм, который обеспечивает корректную работу модульной системы ЖёПки. В этом разделе подробно описаны семантика, варианты использования и ошибки, связанные с самоимпортом.

3.1. Семантика

При загрузке файла интерпретатор читает первую непустую строку и проверяет, что она соответствует шаблону «жёпка принадлежит <идентификатор>.ж». Затем он сравнивает указанное имя с именем файла на диске. Если они совпадают, модуль регистрируется в глобальной таблице модулей с указанным именем. Если модуль уже зарегистрирован, возвращается существующая ссылка — это защищает от двойной загрузки. Только после успешной регистрации интерпретатор приступает к парсингу остального файла.

3.2. Варианты использования

Самоимпорт позволяет программам ссылаться на собственные определения через специальную переменную «я». Например, функция может вызывать саму себя через «я.<имя>», что особенно полезно при рекурсивных вызовах, когда локальное имя может быть затенено параметром. Этот механизм также облегчает отладку: при возникновении ошибки интерпретатор всегда знает, в каком модуле она произошла, поскольку каждый файл явно идентифицирует себя.

```
▶ utils.ж

жёпка принадлежит utils.ж
функция двойной x
3 вернуть x * 2
функция четверной x
3 вернуть я.двойной я.двойной x
```

Пример 3.1: Использование переменной «я» для обращения к собственным функциям через модульный интерфейс.

3.3. Ошибки самоимпорта

Существует три типа ошибок, связанных с самоимпортом. «ОшибкаСамоИмпорта: отсутствует строка самоимпорта» возникает, если файл не начинается с корректной строки. «ОшибкаСамоИмпорта: имя файла не совпадает» — если указанное имя отличается от реального имени файла. «ОшибкаСамоИмпорта: недопустимое имя» — если имя содержит запрещённые символы (заглавные буквы латиницы, дефисы, точки внутри имени, спецсимволы). Все три ошибки фатальны: программа не запускается до их исправления.

4. Вложенность через «3»

Вложенность через «3» — самая узнаваемая черта ЖёПки. В этом разделе подробно описаны правила работы с ней, базовые принципы и переходы между уровнями.

4.1. Базовое правило

Каждая строка исходного кода может начинаться с последовательности символов «3». Количество этих символов определяет уровень вложенности выражения в данной строке относительно предыдущей строки нулевого уровня. После последовательности троек ставится ровно один пробел, затем следует выражение. Строки без маркера находятся на нулевом (верхнем) уровне.

```
▶ базовое.ж

функция квадрат x
3 вернуть x * x
```

*Пример 4.1: Строка «вернуть x * x» находится на уровне 1, поскольку она является телом функции.*

4.2. Многоуровневая вложенность

Для глубоких уровней количество троек увеличивается. Каждый уровень соответствует одной «3» в начале строки. Парсер строго проверяет, что уровни увеличиваются пошагово: переход с уровня N сразу на уровень N+2 без промежуточного уровня N+1 вызывает синтаксическую ошибку «ОшибкаВложенности».

```
▶ многоуровень.ж

функция обработать данные
3 для элемент в данные
33 если элемент > 0
333 печать элемент
3333 вернуть элемент * 2
3 иначе
33 печать "неположительный"
```

Пример 4.2: Четырёхуровневая вложенность: функция → цикл → условие → печать → возврат.

4.3. Правила перехода между уровнями

Переход с уровня N на уровень N+1 разрешён в любой момент — это открывает новый блок. Переход с уровня N на уровень M, где M меньше N, также разрешён в любой момент — это закрывает блоки. Однако переход с уровня N на уровень N+K, где K больше 1, запрещён: уровни должны увеличиваться пошагово. На одном уровне может быть сколько угодно строк подряд. После последовательности «3» должен стоять ровно один пробел; табуляция и несколько пробелов вызывают ошибку «ОшибкаФорматирования».

> **ЧАСТАЯ ОШИБКА**
> Переход с уровня 1 сразу на уровень 3 — самая частая синтаксическая ошибка. Если вам нужно углубиться на 2 уровня, всегда вставляйте промежуточную строку с соответствующим маркером.

4.4. Что считается блоком

Блок — это последовательность строк с одинаковым или большим уровнем вложенности, следующая за заголовком управляющей конструкции. Заголовками являются: «функция», «если», «иначе», «иначе_если», «пока», «для», «попытка», «перехват», «наконец», «класс», «пусть», «тип», «сопоставить», «лямбда» (многострочная форма), «эффект». Каждый из этих заголовков открывает новый блок, и его содержимое должно находиться на уровне N+1, где N — уровень заголовка.

5. Операторы и ключевые слова

В ЖёПке версии 1.0.0 определено 47 ключевых слов, разделённых на четыре категории: базовые (управляющие конструкции и логика), функциональные (высшие функции и связывания), типы данных (встроенные и специальные), продвинутые (исключения, классы, файлы). Все ключевые слова являются дословным переводом соответствующих операторов Python. Полная таблица операторов с переводами, описаниями и примерами использования приведена ниже.

5.1. Базовые операторы

Базовые операторы включают управляющие конструкции (если, иначе, пока, для), определения функций (функция, вернуть), логические связки (и, или, не) и литералы (истина, ложь, ничто). Это минимальный набор для написания любой программы. Перевод прямолинеен: каждое английское слово заменяется его русским эквивалентом, а семантика полностью соответствует Python.

5.2. Таблица операторов

Ниже приведена сокращённая таблица операторов. Полная версия доступна в файле 03_operators.csv датасета.

Python	ЖёПка	Категория	Описание
`print`	`печать`	Базовые	Вывод значения в stdout
`if`	`если`	Базовые	Условный оператор
`else`	`иначе`	Базовые	Альтернативная ветвь
`elif`	`иначе_если`	Базовые	Каскадное условие

Python	ЖёПка	Категория	Описание
while	пока	Базовые	Цикл с предусловием
for	для	Базовые	Цикл по последовательности
def	функция	Базовые	Определение функции
return	вернуть	Базовые	Возврат значения
True	истина	Базовые	Логическое истина
False	ложь	Базовые	Логическое ложь
None	ничто	Базовые	Отсутствие значения
and	и	Базовые	Логическое И
or	или	Базовые	Логическое ИЛИ
not	не	Базовые	Логическое НЕ
map	карта	Функциональные	Применение функции к элементам
filter	функция	Функциональные	Фильтрация по предикату
reduce	свёртка	Функциональные	Свёртка в одно значение
lambda	лямбда	Функциональные	Анонимная функция
let (ML)	пусть	Функциональные	Неизменяемое связывание
generator	ряд	Функциональные	Ленивая последовательность
list	список	Типы_данных	Изменяемая последовательность
tuple	кортеж	Типы_данных	Неизменяемая последовательность
dict	словарь	Типы_данных	Отображение ключ-значение
set	множество	Типы_данных	Множество уникальных элементов
int	целое	Типы_данных	Целое число
float	дробное	Типы_данных	Число с плавающей точкой
str	строка	Типы_данных	Unicode-строка
bool	логическое	Типы_данных	Логический тип
raise	исключение	Продвинутые	Возбуждение исключения
try	попытка	Продвинутые	Блок обработки исключений
except	перехват	Продвинутые	Перехват исключения
finally	наконец	Продвинутые	Блок finally
class	класс	Продвинутые	Определение класса
extends	наследует	Продвинутые	Наследование
import	импорт	Продвинутые	Импорт модуля

Python	ЖёПка	Категория	Описание
open	открыть	Продвинутые	Открытие файла
with	с_файлом	Продвинутые	Контекстный менеджер

Таблица 5.1: Сокращённая таблица ключевых слов Python и их перевода на ЖёПку. Полная версия в файле 03_operators.csv.

6. Типы данных

ЖёПка поддерживает восемь встроенных типов данных и четыре специальных. Восемь встроенных типов покрывают все базовые потребности: числа (целое, дробное), логические значения (логическое), строки (строка) и коллекции (список, кортеж, словарь, множество). Четыре специальных типа — «ячейка» (изменяемая ссылка), «вариант» (конструктор алгебраического типа), «тип» (определение нового типа), «функция» (тип функций первого класса).

6.1. Встроенные типы

Все встроенные типы имеют прямые аналоги в Python, что облегчает переход. Существенное отличие: списки по умолчанию создаются неизменяемыми (эквивалент кортежа Python); для изменяемого списка нужно явно использовать «изменяемый_список». Это решение принято в пользу функциональной чистоты: иммутабельные коллекции по умолчанию упрощают рассуждение о коде и предотвращают случайные мутации.

6.2. Таблица типов

Тип	Python аналог	Мутируемый	Описание
целое	int	Нет	Целое число произвольной точности
дробное	float	Нет	Число с плавающей точкой
логическое	bool	Нет	Логическое значение (истина/ложь)
строка	str	Нет	Unicode-строка
список	list	Да	Изменяемая последовательность
кортеж	tuple	Нет	Неизменяемая последовательность
словарь	dict	Да	Отображение ключ-значение
множество	set	Да	Множество уникальных элементов
ячейка	ref (ML)	Да	Изменяемая ссылка для состояния
вариант	data ctor	Нет	Конструктор варианта АТД

Тип	Python аналог	Мутируемый	Описание
`тип`	`type`	Нет	Определение алгебраического типа
`функция`	`function`	Нет	Тип функций первого класса

Таблица 6.1: Все типы данных ЖёПки. Колонка «Мутируемый» показывает, можно ли изменить значение после создания.

6.3. Статическая vs динамическая типизация

По умолчанию ЖёПка динамически типизирована, как Python. Однако поддерживается опциональная статическая типизация через аннотации: «пусть x целое = 5» или «функция квадрат x целое вернуть целое». Аннотации проверяются во время выполнения (runtime type checking), а не во время компиляции. Полная статическая проверка типов планируется в версии 2.0 с введением эффектной системы.

7. Функциональные возможности

ЖёПка поощряет функциональный стиль программирования. Функции являются значениями первого класса, поддерживаются замыкания, каррирование (через лямбда с несколькими телами), алгебраические типы данных (через тип и вариант), сопоставление с образцом (через сопоставить), а также ленивые последовательности (через ряд). Иммутабельность по умолчанию означает, что «пусть x = 5» создаёт неизменяемую связь; попытка переприсвоить x вызывает «ОшибкаНеизменности».

7.1. Каррирование и частичное применение

Любую функцию нескольких аргументов можно частично применить, передав меньшее число аргументов. Результат — новая функция, ожидающая оставшиеся аргументы. Каррирование работает автоматически для всех функций, объявленных через «функция». Анонимные функции через «лямбда» также поддерживают каррирование. Это позволяет создавать специализированные функции из общих без явного объявления обёрток.

```
▶ карр.ж

жёпка принадлежит карр.ж
# Currying: pass fewer args, get a new function
функция добавить a b
3 вернуть a + b
пусть добавить_5 = добавить 5
печать добавить_5 10   # 15
```

7.2. Высшие функции: карта, функция, свёртка

Три главных оператора высшего порядка — «карта» (применяет функцию к каждому элементу), «функция» (фильтрует по предикату), «свёртка» (сворачивает последовательность в одно значение). Для лучшей читаемости кода рекомендуется использовать именно эти операторы вместо явных циклов «для». Цепочка «функция → карта → свёртка» заменяет цикл с условием и аккумулятором, и код читается как декларация намерения: «взять чётные, возвести в квадрат, сложить».

▶ высший.ж

```
жёпка принадлежит высший.ж
# Filter even, square, then sum: 2^2 + 4^2 = 20
функция чётное n
З вернуть n % 2 == 0
функция сложить a b
З вернуть a + b
пусть числа = список 1 2 3 4 5
пусть чётные = функция чётное числа
пусть кв = карта лямбда x x * x чётные
пусть сумма = свёртка сложить 0 кв
печать сумма   # 20
```

7.3. Лямбда-выражения и замыкания

Анонимные функции объявляются через «лямбда» — аналог lambda в Python. Лямбды удобны для коротких операций и часто используются как аргументы высших функций. Лямбды в ЖёПке поддерживают замыкания: они захватывают переменные из окружающей области видимости. Это позволяет создавать функции-фабрики, которые «запоминают» значения из контекста определения.

▶ замыкание.ж

```
жёпка принадлежит замыкание.ж
# Lambda capturing closure variable
пусть множитель = 10
пусть умножить_на = лямбда x x * множитель
печать умножить_на 5   # 50
```

8. Формальная грамматика

В этом разделе приведены ключевые продукции расширенной BNF-грамматики ЖёПки. Полная грамматика в JSON-формате доступна в файле 04_grammar.json датасета. Грамматика описывает синтаксис верхнего уровня, правила вложенности через символ «З», лексическую структуру и AST-узлы.

8.1. Лексические правила

Лексер ЖёПки распознаёт 47 ключевых слов, 14 операторов, два специальных символа («З» и «#»), пять видов литералов (целое, дробное, строка, булево, nothing) и идентификаторы. Идентификаторы могут содержать буквы кириллицы [а-яё] или латиницы [a-zA-Z], цифры и знак подчёркивания; не могут начинаться с цифры. Комментарии начинаются с «#» и продолжаются до конца строки; они не влияют на уровни вложенности.

8.2. Ключевые продукции

№	Нетерминал	Правило
1	<программа>	<самоимпорт> { <определение> } [<главное>]
2	<самоимпорт>	'жёпка' 'принадлежит' <имя> '.ж'
3	<строка>	[<маркер_вложенности>] <содержимое>
4	<маркер>	'З' { 'З' } — количество = уровень
5	<функция>	'функция' <имя> { <парам> } <тело>
6	<параметр>	<имя> [<тип>] ['=' <умолчание>]
7	<вызов>	<имя> { <аргумент> } — каррирование
8	<условие>	'если' <усл> <тело> ['иначе_если' <усл> <тело>]* ['иначе' <тело>]
9	<цикл_для>	'для' <перем> 'в' <итератор> <тело>
10	<пусть>	'пусть' <имя> [<тип>] '=' <выражение>
11	<лямбда>	'лямбда' { <парам> } <выражение>
12	<блок_try>	'попытка' <тело> { 'перехват' <тип> 'как' <имя> <тело> } ['наконец' <тело>]
13	<класс>	'класс' <имя> ['наследует' <родитель>] <тело_класса>
14	<тип_АТД>	'тип' <имя> { 'вариант' <имя> [{ <поле> }] }
15	<сопоставить>	'сопоставить' <выражение> { <образец> 'вернуть' <выражение> }

Таблица 8.1: Ключевые продукции EBNF-грамматики ЖёПки. Полная грамматика в файле 04_grammar.json.

8.3. Правила вложенности

Правила вложенности через «З» — самая узнаваемая черта ЖёПки. Каждая строка может начинаться с последовательности символов «З», и количество этих символов определяет уровень вложенности. Переходы между уровнями должны быть пошаговыми: нельзя перепрыгивать через уровни. После последовательности «З» должен стоять ровно один пробел. Полностью пустая строка (без «З») интерпретируется как конец всех открытых блоков — это может привести к неожиданным ошибкам парсинга, поэтому рекомендуется либо не использовать пустые строки внутри блоков, либо явно ставить соответствующее количество троек.

9. Примеры кода

В этом разделе приведены двенадцать ключевых примеров кода на ЖёПке. Код пишется на русском (поскольку весь синтаксис языка русский), а комментарии к коду — на английском. Полный сборник из 22 примеров доступен в файле 06_examples.docx датасета.

9.1. Hello World — Привет, мир

Минимальная программа: самоимпорт и вывод строки в стандартный поток. Никаких точек входа main, никакой неявной обёртки — код верхнего уровня выполняется последовательно сверху вниз.

```
▶ hello.ж

жёпка принадлежит привет.ж
# Print the classic greeting to standard output
печать "Привет, мир!"
```

9.2. Recursive Factorial — Факториал

Классическая рекурсивная функция факториала. Базовый случай n ≤ 1 возвращает 1; рекурсивный случай умножает n на факториал от n-1. Это прямая рекурсия — для больших n лучше использовать хвостовую рекурсию с аккумулятором.

```
▶ factorial.ж

жёпка принадлежит факториал.ж
# Classic factorial using direct recursion
функция факториал n
3 если n <= 1
33 вернуть 1
3 вернуть n * факториал (n - 1)

# Compute 5! = 120
печать факториал 5
```

9.3. Fibonacci Sequence — Числа Фибоначчи

Наивная реализация Фибоначчи через прямую рекурсию. Экспоненциальная сложность O(2^n) делает её непрактичной для n > 30. Для больших значений используйте мемоизацию или итеративный подход.

```
▶ fibonacci.ж

жёпка принадлежит фибоначчи.ж
# Naive O(2^n) Fibonacci via direct recursion
функция фибоначчи n
3 если n < 2
33 вернуть n
3 вернуть фибоначчи (n - 1) + фибоначчи (n - 2)

# Print first 10 Fibonacci numbers: 0 1 1 2 3 5 8 13 21 34
для i в диапазон 0 10
3 печать фибоначчи i
```

9.4. Map — Squaring Numbers

Применение функции к каждому элементу списка через оператор «карта». Лямбда-выражение «лямбда x x * x» определяет анонимную функцию возведения в квадрат.

```
▶ map_square.ж

жёпка принадлежит карта_квадрат.ж
# Use "карта" (map) to square every number in a list
пусть числа = список 1 2 3 4 5
пусть квадраты = карта лямбда x x * x числа

# Output: [1, 4, 9, 16, 25]
печать квадраты
```

9.5. Filter — Even Numbers

Отбор только чётных элементов списка. Оператор «функция» (перевод filter, не def) принимает предикат и последовательность, возвращая новую последовательность с элементами, для которых предикат истинен.

```
▶ filter_even.ж

жёпка принадлежит фильтр_чётных.ж
# Use "функция" (filter) to keep only even numbers
функция чётное n
3 вернуть n % 2 == 0

пусть числа = список 1 2 3 4 5 6 7 8 9 10
пусть чётные = функция чётное числа

# Output: [2, 4, 6, 8, 10]
печать чётные
```

9.6. Reduce — Sum of Elements

Суммирование всех элементов списка через свёртку. Оператор «свёртка» принимает функцию двух аргументов, начальное значение аккумулятора и последовательность. На каждом шаге аккумулятор обновляется результатом вызова функции.

```
▶ reduce_sum.ж

жёпка принадлежит свёртка_сумма.ж
# Use "свёртка" (reduce) to compute the sum
функция сложить a b
3 вернуть a + b

пусть числа = список 1 2 3 4 5 6 7 8 9 10
пусть сумма = свёртка сложить 0 числа

# Output: 55
печать сумма
```

9.7. Quicksort — Быстрая сортировка

Quicksort через рекурсию и фильтрацию. Выбираем опорный элемент (первый), делим список на меньшие и большие/равные, рекурсивно сортируем обе части, соединяем. Это классический функциональный quicksort.

```
▶ quicksort.ж

жёпка принадлежит qsort.ж
# Quicksort using recursion and filter
функция быстрая_сортировка lst
3 если длина lst <= 1
33 вернуть lst
3 пусть опорный = первый lst
3 пусть остальное = хвост lst
3 пусть меньше = функция лямбда x x < опорный остальное
3 пусть больше_или_равно = функция лямбда x x >= опорный остальное
3 вернуть (быстрая_сортировка меньше) + [опорный] + (быстрая_сортировка больше_или_равно)

пусть данные = список 5 3 8 1 9 2 7 4 6
печать быстрая_сортировка данные   # [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

9.8. Towers of Hanoi — Ханойские башни

Классическая задача о ханойских башнях с рекурсивным решением. Базовый случай: один диск перемещается напрямую. Рекурсивный случай: переносим n-1 дисков с источника на промежуточный, затем переносим оставшийся диск на цель, затем переносим n-1 дисков с промежуточного на цель.

```
▶ hanoi.ж

жёпка принадлежит ханой.ж
# Classic Towers of Hanoi — recursive solution
функция ханой n источник промежуток цель
3 если n == 1
33 печать "Move disk 1 from " + источник + " to " + цель
33 вернуть
3 ханой (n - 1) источник цель промежуток
3 печать "Move disk " + n + " from " + источник + " to " + цель
3 ханой (n - 1) промежуток источник цель

# Solve for 3 disks: A -> C using B as auxiliary
ханой 3 "A" "B" "C"
```

9.9. Binary Tree Traversal — Обход дерева

Определение бинарного дерева как алгебраического типа данных и обход в порядке in-order (лево-корень-право). Pattern matching через «сопоставить» обрабатывает оба варианта (Пусто и Узел) с деструктуризацией полей.

```
▶ tree.ж

жёпка принадлежит дерево.ж
# Define an algebraic data type for a binary tree
тип Дерево
3 вариант Пусто
3 вариант Узел Дерево целое Дерево

# In-order traversal: left, root, right
функция обход дерево
3 сопоставить дерево
33 Пусто вернуть []
33 Узел лево значение право
333 вернуть (обход лево) + [значение] + (обход право)

# Build sample tree:    2
#                      / \
#                     1   3
пусть дерево = Узел (Узел Пусто 1 Пусто) 2 (Узел Пусто 3 Пусто)
печать обход дерево   # [1, 2, 3]
```

9.10. Binary Search — Двоичный поиск

Классический бинарный поиск в отсортированном массиве. Итеративный подход с изменяемыми границами через «ячейка». Возвращает индекс элемента или -1, если элемент не найден. Сложность O(log n).

```
▶ binary_search.ж

жёпка принадлежит бпоиск.ж
# Iterative binary search on a sorted array
функция двоичный_поиск lst цель
3 пусть нижняя ячейка = 0
3 пусть верхняя ячейка = длина lst - 1
3 пока !нижняя <= !верхняя
33 пусть середина = (!нижняя + !верхняя) / 2
33 если lst[середина] == цель
333 вернуть середина
33 иначе_если lst[середина] < цель
333 нижняя := середина + 1
33 иначе
333 верхняя := середина - 1
3 вернуть -1

пусть данные = список 1 3 5 7 9 11 13 15 17 19
печать двоичный_поиск данные 11   # 5 (index)
печать двоичный_поиск данные 8    # -1 (not found)
```

9.11. Read File — Чтение файла

Чтение содержимого текстового файла через контекстный менеджер «с_файлом». Файл автоматически закрывается при выходе из блока, что предотвращает утечку ресурсов. Использование «ячейка» позволяет накопить результат внутри блока.

```
▶ read_file.ж

жёпка принадлежит чтение.ж
# Read entire file content using context manager
функция прочитать_файл путь
3 пусть содержимое ячейка = ""
3 с_файлом f = открыть путь "чтение"
33 содержимое := читать f
3 вернуть !содержимое

пусть текст = прочитать_файл "input.txt"
печать текст
```

9.12. Write Log File — Запись лога

Запись сообщений в лог-файл в режиме добавления. Файл открывается вручную через «открыть», записывается через «писать», явно закрывается через «закрыть». Режим «добавление» (append) сохраняет существующее содержимое файла и дописывает новые строки в конец.

```
▶ log.ж

жёпка принадлежит лог.ж
# Append a log message with manual file close
функция записать_лог сообщение
3 пусть f = открыть "app.log" "добавление"
3 писать f "[" + сейчас + "] " + сообщение + "\n"
3 закрыть f

записать_лог "Приложение запущено"
записать_лог "Пользователь авторизован"
записать_лог "Завершение работы"
```

10. Обработка ошибок

Ошибки обрабатываются через конструкцию «попытка ... перехват ... наконец» — перевод try/except/finally. Возбуждение исключений — через «исключение» (аналог raise). В ЖёПке предопределены тринадцать типов исключений, покрывающих наиболее частые сценарии: деление на ноль, выход за границы, отсутствие ключа, неверный тип, и другие. Пользовательские исключения объявляются через «класс ... наследует Ошибка».

10.1. Синтаксис обработки

```
▶ ошибки.ж

жёпка принадлежит ошибки.ж
# Safe division with error handling
функция безопасное_деление a b
3 попытка
33 вернуть a / b
3 перехват ОшибкаДеления как e
33 печать "Деление на ноль!"
33 вернуть ничто
3 наконец
33 печать "Деление завершено"
```

Пример 10.1: Полная конструкция попытка/перехват/наконец с привязкой исключения к переменной.

10.2. Таблица ошибок

Имя ошибки	Категория	Когда возникает
ОшибкаСамоИмпорта	Системная	Нарушение правил самоимпорта файла
ОшибкаВложенности	Синтаксическая	Пропуск уровня вложенности или некорректный переход
ОшибкаФорматирования	Синтаксическая	Несколько пробелов после «3», табуляция вместо пробела

Имя ошибки	Категория	Когда возникает
ОшибкаСинтаксиса	Синтаксическая	Неверный токен, незакрытая строка, нарушение грамматики
ОшибкаНеизменности	Runtime	Попытка переприсвоить связь «пусть»
ОшибкаТипа	Runtime	Несоответствие типов в операции
ОшибкаДеления	Runtime	Деление на ноль или остаток от деления на ноль
ОшибкаИндекса	Runtime	Обращение к элементу по индексу вне границ
ОшибкаКлюча	Runtime	Обращение к словарю по отсутствующему ключу
ОшибкаЗначения	Runtime	Аргумент правильного типа, но недопустимого значения
ОшибкаИмени	Runtime	Обращение к несуществующему имени в области видимости
ОшибкаАтрибута	Runtime	Обращение к несуществующему атрибуту объекта
ОшибкаПереполнения	Runtime	Превышение максимальной глубины рекурсии
ОшибкаФайла	Runtime	Ошибка файловой операции (нет файла, нет прав)

Таблица 10.1: Тринадцать встроенных типов ошибок ЖёПки. Все могут быть перехвачены через «перехват».

10.3. Пользовательские исключения

Для создания собственных типов исключений используется наследование от встроенного типа «Ошибка». Пользовательское исключение определяется через «класс ... наследует Ошибка» и может содержать собственные поля и методы. При возбуждении исключения через «исключение <тип> <сообщение>» интерпретатор создаёт экземпляр указанного типа и передаёт его в блок «перехват».

```
▶ кастом_ошибки.ж

жёпка принадлежит кастом_ошибки.ж
# Custom exception type
class ОшибкаВозраста наследует Ошибка
3 функция __init__ возраст
33 вернуть "Недопустимый возраст: " + возраст

функция проверить_возраст n
3 если n < 0 или n > 150
33 исключение ОшибкаВозраста n
3 вернуть "ОК"

попытка
3 проверить_возраст -5
перехват ОшибкаВозраста как е
3 печать е   # Недопустимый возраст: -5
```

Пример 10.2: Создание и перехват пользовательского исключения ОшибкаВозраста.

11. Дорожная карта и приложения

11.1. Дорожная карта развития

Текущая версия 1.0.0 определяет ядро языка. В будущих версиях планируется расширение функциональности с сохранением обратной совместимости. Основные направления развития: ленивые последовательности, алгебраические типы данных, эффектная система, параллельные вычисления, и компилятор в нативный код.

Версия	Ключевые возможности	Статус
1.0.0	Базовое ядро: 47 ключевых слов, 12 типов, грамматика	Stable
1.0.1	Исправление багов парсера, оптимизация хвостовой рекурсии	In progress
1.1.0	Ленивые «ряд», pattern matching «сопоставить», АТД	Planned
1.2.0	Полная статическая типизация, type inference	Planned
2.0.0	Эффектная система, формализация I/O, сетевые операции	Research
2.1.0	Параллельные вычисления: «асинхронно», «ждать», «задача»	Research
3.0.0	Компилятор в LLVM, нативный код, оптимизации	Long-term

Таблица 11.1: Дорожная карта развития языка ЖёПка.

11.2. Структура датасета документации

Этот PDF — часть датасета документации ЖёПки из восьми файлов в разных форматах. Каждый формат содержит логически обособленный раздел документации: введение, FAQ, операторы, грамматика, туториал, примеры, таблицы и спецификация.

№	Файл	Формат	Назначение
1	01_introduction.md	Markdown	Введение в язык и философия
2	02_faq.txt	TXT	Часто задаваемые вопросы
3	03_operators.csv	CSV	Таблица операторов Python → ЖёПка
4	04_grammar.json	JSON	Формальная BNF-грамматика
5	05_tutorial.html	HTML	Туториал с примерами
6	06_examples.docx	DOCX	Сборник примеров кода
7	07_tables.xlsx	EXCEL	Детальные таблицы

№	Файл	Формат	Назначение
8	08_specification.pdf	PDF	Этот документ — полная спецификация

Таблица 11.2: Восемь файлов датасета документации ЖёПки в разных форматах.

11.3. Заключение

ЖёПка — это эксперимент, объединяющий три редко совместимых идеи: русскоязычный синтаксис, отказ от скобок в пользу явного числового маркера вложенности и строгую функциональную дисциплину. Язык предназначен прежде всего для исследовательских и образовательных целей, однако его спецификация достаточно полна для реализации интерпретатора или компилятора. Дополнительные материалы — формальная грамматика, таблицы операторов, туториал с примерами и сборник готовых программ — доступны в сопутствующих файлах датасета в форматах JSON, CSV, HTML, TXT, DOCX и XLSX.

Спецификация версии 1.0.0 стабильна: будущие версии будут добавлять новые возможности, но не изменять существующие продукции. Это гарантирует, что программы, написанные на ЖёПке 1.0.0, продолжат работать в будущих версиях без модификаций. ЖёПка не претендует на продакшен-применимость в версии 1.x; его цель — исследование альтернативных синтаксических и семантических решений в функциональной парадигме.